

## Mini project 1 in Matrix Theory, HT 2026

The mini project is best to do in groups of two persons, but you can do it alone as well.

The task First write a Python script which decides if a given (integer) matrix  $A$  has integer eigenvalues and, if this is the case, calculates those eigenvalues and their multiplicities. The function call should look like:

```
def heltalsev(A, tol=0.0001):
```

Here  $ev$  is a vector which contains the distinct eigenvalues and  $mult$  is a vector with the corresponding multiplicities. With this construction, it is sufficient to call the program with `heltalsev(A)`. The syntax `tol=0.0001` above makes the `tol` parameter optional with a default value of 0.0001, but without changing the program it is possible to alter the tolerance by instead calling `heltalsev(A,nytol)`. For this task, the `numpy` and `scipy` packages are very useful, which you can include using:

```
import numpy as np
import scipy
```

For calculations of eigenvalues, the function `np.linalg.eigvals` is useful, and to decide if they are integers, functions like `np.round`, `np.linalg.norm`, and others may be useful. Besides that, some of the functions like `np.unique` (with the parameter `return_counts=True`) may be useful. After you are done with the function `heltalsev` write a function which, for a matrix  $A$  with integer eigenvalues, calculates the Jordan normal form. The result should be a Jordan matrix of the same size as  $A$ . The first lines should be

```
def jordanmatrix(A, tol=0.00001):
    ev, mult = heltalsev(A, tol)
```

Use the method in Chapter 7.3, 7.5. Here, built-in functions such as `np.linalg.matrix_rank`, `scipy.linalg.null_space`, and `np.diff` may be of use. To construct the Jordan matrix, functions such as `np.diag`, `np.eye`, `np.zeros`, `np.ones`, and `scipy.linalg.block_diag` can be used.

Testing Test your program on many different matrices to make sure that it works properly. Some examples can be found in the `testmatrices.py` file provided from the home page. By placing your Python script in the same folder as `testmatrices.py`, you can retrieve an example matrix using the code below. Replace `n` by any number between 0 and 6 get that matrix. Some of these example matrices will be explained further below.

```
from testmatrices import testmatrix
A = testmatrix(n)
```

To think about When you calculate eigenvalues with `np.linalg.eigvals`, note that this is a complex command. Python computes the eigenvalues using about 16 decimals, but the error in the computed eigenvalues may be much larger. If the difference to the closest integer is smaller than your tolerance, you should round to the closest integer before constructing the Jordan form. Choosing a reasonable favorite tolerance for `tol` is part of the problem. To estimate how large the error can be, try to calculate the eigenvalues of

$$\begin{pmatrix} -9 & 11 & -21 & 63 & -252 \\ 70 & -69 & 141 & -421 & 1684 \\ -575 & 575 & -1149 & 3451 & -13801 \\ 3891 & -3891 & 7782 & -23345 & 93365 \\ 1024 & -1024 & 2048 & -6144 & 24572 \end{pmatrix}$$

which only has a size of  $5 \times 5$  and has 0 as the only eigenvalue (with multiplicity). This matrix can be retrieved from `testmatrix(3)` from the `testmatrices.py` script. The matrix above is exceptional, and you should not choose your tolerance based on it alone. If you do, it is likely that your program will accept the matrices that have no integer eigenvalues, such as

$$\begin{pmatrix} 1000 & 1 \\ 1 & 0 \end{pmatrix}.$$

Thus, there is no universal tolerance, and that is why we have an option to use the tolerance as a parameter.

Final test When you feel that the program is working on small examples, check it on a larger  $10 \times 10$  matrix, found in `testmatrix(4)`. It is internally constructed using the following code

```
coeffs = np.poly(np.arange(1, 11))
A = np.polynomial.polynomial.polycompanion(coeffs[::-1])
```

This constructs the companion matrix to the polynomial with roots in  $1, 2, \dots, 10$ , which we know has eigenvalues exactly  $1, 2, \dots, 10$ . (The command `coeffs[::-1]` is required to reverse the order of the coefficients as `np.poly` returns coefficients in descending order, while `polycompanion` wants them in ascending order.)

If your program crashes, it is probably due to unnecessary matrix multiplication (high powers of large matrices, such as `A` above, contain the numbers that are larger than  $10^{16}$ , which results in approximation errors). Thus, you need to reduce the number of

multiplications, especially when the multiplicity is low. Often, you need only small modifications.

If you did not use that the eigenvalues are real, it is possible that your program works with Gauss numbers (complex integers) as in `testmatrix(5)`:

$$\begin{pmatrix} 3 & -4 \\ 4 & 3 \end{pmatrix}.$$

**Sending** Put both your functions, one after the other, in a single file. Give it the name `jordanmatrix.py` and upload it through the Canvas page. Check that the program is well commented and easy to follow. If you work in pairs both of you still have to submit separately in canvas. However, you can both hand in exactly the same files. Write in the comments to the submission who you have worked with.

Good luck!